Vue 컴포넌트 통신 개념 정리

컴포넌트 통신 방향

```
부모 → 자식 : props
자식 → 부모 : emit
형제 / 비관계 : mitt (Event Bus)
```

---

1. Single File Component (SFC)

Vue는 하나의 `.vue` 파일 안에 HTML, JavaScript, CSS를 함께 작성할 수 있다.
이를 **Single File Component(SFC)** 라고 한다.

Options API (수업 기준)

```vue
<template>
  <h1>{{ title }}</h1>
</template>

<script>
export default {
  data() {
    return {
      title: 'Hello Vue!'
    }
  }
}
</script>

<style scoped>
h1 {
  color: blue;
}
</style>
```

구성요소	역할
`<template>`	화면 구조 (HTML)
`<script>`	로직 (JavaScript)
`<style scoped>`	CSS — `scoped`를 붙이면 이 컴포넌트에만 적용

> `scoped`를 붙이지 않으면 전체 애플리케이션에 CSS가 적용된다.

> **참고 — Composition API (`<script setup>`)**
> Vue 3에서 공식 권장하는 방식
>
> ```vue
> <template>
>  <h1>{{ title }}</h1>
> </template>
>
> <script setup>
> import { ref } from 'vue'
>
> const title = ref('Hello Vue!')
> </script>
>
> <style scoped>
> h1 { color: blue; }
> </style>
> ```

---

2. 컴포넌트 트리와 부모-자식 관계

Vue 애플리케이션은 여러 컴포넌트를 조합하여 구성한다.
이 구조를 **컴포넌트 트리**라고 한다.

```
App.vue (루트)
 ├── CheckboxItem.vue
 ├── CheckboxItem.vue
 └── CheckboxItem.vue
```

`<template>` 안에 다른 컴포넌트를 사용하는 구조가 **부모-자식 관계**이다.

```vue
```

```
<!-- App.vue → 부모 -->
<template>
  <CheckboxItem />  <!-- CheckboxItem → 자식 -->
</template>
```

컴포넌트 등록 방법

전역 등록 — 어느 컴포넌트에서든 사용 가능하지만, 불필요한 파일까지 포함될 수 있다.

```js
// main.js
import CheckboxItem from './components/CheckboxItem.vue'
app.component('CheckboxItem', CheckboxItem)
```

지역 등록 (권장) — 필요한 컴포넌트 안에서만 import하여 사용한다.

```vue
<!-- Options API -->
<script>
import CheckboxItem from './components/CheckboxItem.vue'

export default {
  components: {
    CheckboxItem
  }
}
</script>
```

> **참고 — Composition API 지역 등록**
> `<script setup>`에서는 import만 하면 자동으로 등록된다.
>
> ```vue
> <script setup>
> import CheckboxItem from './components/CheckboxItem.vue'
> // components 옵션 없이 바로 사용 가능
> </script>
> ```

---

3. Props — 부모 → 자식

부모가 자식에게 데이터를 전달하는 방법이다.

핵심 규칙

- 부모가 데이터를 소유한다
- props는 **읽기 전용**이다 (자식에서 직접 수정 불가)
- 자식이 변경이 필요하면 **emit**을 사용한다

문법

부모 — 전달

```vue
<자식컴포넌트 :속성명="값" />
```

자식 — 수신

```vue
<!-- Options API -->
props: ['속성명']

props: {
  속성명: { type: 타입, required: true, default: 기본값 }
}
```

```vue
<!-- Composition API -->
const props = defineProps({ 속성명: { type: 타입 } })
```

예시

```vue
<!-- 부모 -->
<CheckboxItem :name="idol.name" :checked="idol.checked" />

<!-- 객체 전체 전달 -->
<CheckboxItem :idol="idol" />
```

```vue
```

```
<!-- 자식 (Options API) -->
export default {
  props: {
    name:    { type: String,  required: true },
    checked: { type: Boolean, default: false }
  }
}
```

```vue
<!-- 자식 (Composition API) -->
const props = defineProps({
  name:    { type: String,  required: true },
  checked: { type: Boolean, default: false }
})
```

---

props 타입 종류

`String` `Number` `Boolean` `Array` `Object` `Date` `Function`

단방향 데이터 흐름

```
부모 (데이터 소유)
  ↓  props (읽기 전용)
자식 (읽기만 가능)
```

---

4. emit — 자식 → 부모

Props가 부모 → 자식 방향이라면, **emit**은 자식 → 부모 방향으로 데이터를 전달한다.

문법

자식 — 이벤트 발생

```vue
```

```
<!-- Options API -->
this.$emit('이벤트명', 전달데이터)

<!-- Composition API -->
const emit = defineEmits(['이벤트명'])
emit('이벤트명', 전달데이터)
```

부모 — 이벤트 수신

```vue
<자식컴포넌트 @이벤트명="핸들러" />
```

예시

```vue
<!-- 자식 (Options API) -->
methods: {
  handleClick() {
    this.$emit('submit-name', this.inputValue)
  }
}
```

```vue
<!-- 자식 (Composition API) -->
const emit = defineEmits(['submit-name'])
emit('submit-name', inputValue.value)
```

```vue
<!-- 부모 -->
<InputName @submit-name="handleName" />

methods: {
  handleName(name) {
    this.receivedName = name
  }
}
```

---

emits 옵션 — 유효성 검증

```vue
<!-- Options API -->
emits: ['이벤트명']                    // 배열 방식 (이름만 선언)

emits: {                               // 객체 방식 (유효성 검사)
  '이벤트명': (value) => {
    if (!value) return false           // false → 경고
    return true                        // true  → 정상
  }
}

<!-- Composition API -->
const emit = defineEmits({
  '이벤트명': (value) => !!value
})
```

---

props + emit 흐름 정리

```
부모 (데이터 소유)
  |
  |  props (읽기 전용 ↓)
  ▼
자식 (데이터 사용)
  |
  |  emit (변경 요청 ↑)
  ▼
부모 (상태 변경)
```

> 자식은 데이터를 직접 수정하지 않는다.
> 변경 요청만 emit으로 전달하고, **부모가 최종적으로 상태를 변경**한다.

---

5. mitt (Event Bus) — 비관계 컴포넌트 통신

Props와 emit은 부모-자식 관계에서만 사용 가능하다.
형제 컴포넌트처럼 직접적인 관계가 없는 경우 **Event Bus** 방식을 사용한다.

> 컴포넌트 계층이 깊어지면 `provide / inject` 또는 **Pinia(전역 상태 관리)** 사용을 권장한다.

통신 구조

```
Sender 컴포넌트
```

```
      |  emitter.emit("message", data)
      ▼
  Event Bus (mitt)
      |  emitter.on("message", handler)
      ▼
Receiver 컴포넌트
```

설치

```bash
npm install mitt
```

---

mitt 기본 메서드

메서드	설명
`emitter.on(event, handler)`	이벤트 구독
`emitter.emit(event, data)`	이벤트 발생
`emitter.off(event, handler)`	이벤트 구독 해제

전역 등록 (main.js)

```js
import mitt from 'mitt'

const emitter = mitt()
app.config.globalProperties.emitter = emitter  // 모든 컴포넌트에서 this.emitter 사용 가
```

예시

```vue
```

```
<!-- Sender.vue (Options API) -->
methods: {
  send() {
    this.emitter.emit('message', '전달할 데이터')
  }
}
```

```vue
<!-- Receiver.vue (Options API) -->
created() {
  this.emitter.on('message', this.handler)     // 구독
},
beforeUnmount() {
  this.emitter.off('message', this.handler)    // 해제
},
methods: {
  handler(msg) { this.text = msg }
}
```

```vue
<!-- Sender / Receiver (Composition API) -->

// src/eventBus.js
import mitt from 'mitt'
export const emitter = mitt()

// Sender.vue
import { emitter } from '@/eventBus'
emitter.emit('message', '데이터')

// Receiver.vue
import { emitter } from '@/eventBus'
onMounted(()       => emitter.on('message', handler))
onBeforeUnmount(() => emitter.off('message', handler))
```

> 컴포넌트가 제거될 때 구독을 해제한다. 해제하지 않으면 **메모리 누수**가 발생한다.

전체 흐름 정리

```
1. Sender.vue
   emitter.emit("message", data)  ← 이벤트 발생
```

```
2. Event Bus (mitt)
   이벤트 전달

3. Receiver.vue
   emitter.on("message", handler)  ← 이벤트 수신

4. handler 실행 → 화면 업데이트
```

---

6. 컴포넌트 통신 방식 비교 정리

상황	방법	비고
부모 → 자식	`props`	읽기 전용
자식 → 부모	`emit`	변경 요청만 전달
형제 / 비관계 컴포넌트	`mitt (Event Bus)`	단순한 경우에 적합
깊은 컴포넌트 트리	`provide / inject`	중간 컴포넌트 생략 가능
전역 상태 관리	`Pinia`	규모가 커질 때 권장

핵심 원칙 5가지

```
1. 부모가 데이터를 소유한다.
2. props는 읽기 전용이다. (자식에서 직접 수정 금지)
3. 자식은 emit으로만 부모에게 변경을 요청한다.
4. 데이터 흐름은 단방향이다. (부모 → 자식)
5. 이벤트 구독은 컴포넌트 제거시 해제한다.
```

Options API vs Composition API 한눈에 비교

항목	Options API (수업 기준)	Composition API (`script setup`)
props 선언	`props: []` / `props: {}`	`defineProps()`
emit 선언	`emits: []` / `emits: {}`	`defineEmits()`
이벤트 발생	`this.$emit()`	`emit()`

항목	Options API (수업 기준)	Composition API (`script setup`)
컴포넌트 등록	`components: {}` 필요	import만으로 자동 등록
라이프사이클	`created` `beforeUnmount`	`onMounted` `onBeforeUnmount`
Vue 권장	Vue 2 / Vue 3 모두 지원	Vue 3 공식 권장

> 📎 참고 문서
>
> - Vue Props
> - Vue emit
> - mitt
> - Pinia